Übung 4

Primitive Datentypen

Programmierung 1 · WS 2026/27
Bachelor Cyber Security & Privacy · André Kless, M.Sc.

Ziel: Werte binär kodieren, mit Java-Datentypen rechnen und Informationsverluste durch Überlauf, Rundung und Typumwandlung erklären.

Arbeitsweise: Bestimmen Sie Ergebnisse zuerst von Hand und prüfen Sie sie anschließend mit Java 25 oder höher. Verwenden Sie im Projekt `Prog1` je Aufgabe eine eigene Datei, z. B. `K04Aufgabe01.java`, mit `void main()`. Die Codeausschnitte gehören jeweils in diese Methode.

Reihenfolge: Basic (1–4) festigt die Grundlagen, Combined (5–8) verbindet sie, Challenge (9–10) untersucht die Multiplikation bis auf die einzelnen Bits. Nutzen Sie die Folien aus Kapitel 4 zum Nachschlagen.

Eingaben: Sie dürfen die jeweils geforderte Form und den angegebenen Wertebereich voraussetzen. Untersuchen Sie absichtlich fehlerhafte Codezeilen getrennt vom übrigen Programm.

01 Wahrheitswerte und Kurzschluss | Basic

a) Erstellen Sie eine Wahrheitstabelle für `!p`, `p && q`, `p || q`, `p ^ q`, `!(p && q)` und `!p || !q`. Berücksichtigen Sie alle vier Kombinationen von `p` und `q`. Vergleichen Sie insbesondere die letzten beiden Spalten: Was fällt Ihnen auf?

b) Setzen Sie in `p || q && !p` Klammern so, dass die Prioritäten von Java sichtbar werden. Welches Ergebnis entsteht für `p = true` und `q = false`?

c) Welchen Wert erhält `gueltig`? Erklären Sie die Auswertung. Was ändert sich, wenn Sie `&&` durch `&` ersetzen? Prüfen Sie beide Varianten.

```
int anzahl = 0;
boolean gueltig = anzahl != 0 && 12 / anzahl > 2;
```

02 Zahlensysteme und Speichergrößen | Basic

a) Stellen Sie die dezimale Zahl 173 im Dualsystem mit acht Bits, im Oktalsystem und im Hexadezimalsystem dar. Dokumentieren Sie die Umrechnung.

b) Rechnen Sie 00110110_2 und $7B_{16}$ in Dezimalzahlen um. Zeigen Sie jeweils die Summe der Ziffern multipliziert mit ihren Stellenwerten.

c) Deklarieren Sie vier `int`-Variablen mit dem Wert 173. Schreiben Sie die Zahl dabei als dezimales, binäres, oktales bzw. hexadezimales Java-Literal. Geben Sie alle vier Werte aus.

d) Für 10 000 Werte werden jeweils acht Bits gespeichert. Wie viele Bits, Byte und Kilobyte sind das insgesamt? Rechnen Sie wie in der Vorlesung mit 1 kB = 1000 Byte und lassen Sie zusätzlichen Verwaltungsaufwand außer Betracht.

Basic: Ganzzahltypen und Rechenregeln

03 Zweierkomplement und passende Typen | Basic

a) Kodieren Sie −37 mit acht Bits im Zweierkomplement. Welche Dezimalzahl stellt umgekehrt die Bitfolge `11010110` als Acht-Bit-Zweierkomplement dar? Zeigen Sie beide Rechenwege.

b) Erweitern Sie die Kodierungen von −37 und +37 von acht auf 16 Bits. Begründen Sie, mit welchen Bits Sie auffüllen. Welche führenden Bits dürfen Sie in einer verkürzten Darstellung weglassen, ohne Wert oder Vorzeichen zu verändern?

c) Wählen Sie jeweils den kleinsten passenden Java-Datentyp aus `byte`, `short`, `int` und `long` für diese Wertebereiche. Begründen Sie Ihre Wahl: −20 bis 50; −20 000 bis 20 000; 0 bis 50 000; 0 bis 3 000 000 000.

04 Rechentyp und Typumwandlung | Basic

Gehen Sie für jede der folgenden sechs Zeilen erneut von `byte a = 90, b = 50;` aus. Die Zeilen werden unabhängig voneinander untersucht.

```
int summe = a + b;
byte klein = a + b;
byte gekuerzt = (byte) (a + b);
double halb = 7 / 2;
double genau = (double) 7 / 2;
a += b;
```

a) Welche Zeilen akzeptiert der Compiler? Geben Sie für jede gültige Zeile den Rechentyp des Ausdrucks und den anschließend gespeicherten Wert an. Begründen Sie auch die Fehlermeldung zur ungültigen Zeile.

b) Vergleichen Sie den expliziten Cast nach `byte` mit der Kurzzuweisung `+=`. Wo geht Information verloren? Verhindert der Typ der Zielvariablen solche Verluste automatisch?

c) Untersuchen Sie den folgenden Code unabhängig von den vorherigen Zeilen. Welche Zahl wird ausgegeben? Erklären Sie getrennt, welchen Wert `a++` liefert, wie es `a` verändert und was die anschließende Zuweisung bewirkt. Wie würden Sie `a` stattdessen verständlich um 1 erhöhen?

```
int a = 4;
a = a++;
IO.println(a);
```

Combined: Bits und Gleitkommazahlen

05 RGB-Farben und Bitoperatoren | Combined

Eine RGB-Farbe besteht aus Rot, Grün und Blau mit jeweils acht Bits und Werten von 0 bis 255. In der Hexadezimalschreibweise `RRGGBB` stehen je zwei Ziffern für einen Kanal. Wir speichern die Farbe in den unteren 24 Bits eines `int`; die oberen acht Bits bleiben 0. Beispiel: `int farbe = 0x3366CC;`.

Rot (RR)	Grün (GG)	Blau (BB)	Beispielfarbe
33	66	CC	0x3366CC

a) Bestimmen Sie die drei Kanalwerte des Beispiels dezimal und binär. Schreiben Sie ein Programm, das sie mit Bitverschiebungen und `&` aus `farbe` gewinnt und ausgibt.

b) Setzen Sie die drei Kanalwerte mit Verschiebungen und `|` wieder zu einer Farbe zusammen. Prüfen Sie, ob der ursprüngliche Wert entsteht.

c) Berechnen Sie die Komplementärfarbe als RGB-Invertierung: Ersetzen Sie jeden Kanalwert durch 255 minus seinen bisherigen Wert. Setzen Sie die neuen Kanäle zusammen und notieren Sie das Ergebnis hexadezimal. Testen Sie auch Schwarz (`0x000000`), Weiß (`0xFFFFFF`) und Rot (`0xFF0000`).

d) Erreichen Sie dieselbe Invertierung direkt mit `~` und einer geeigneten Bitmaske. Warum reicht `~farbe` allein für unsere 24-Bit-Farbe nicht aus?

e) Verknüpfen Sie die ursprüngliche Farbe mit `int schluessel = 0xA5C39F;` durch XOR. Stellen Sie die ursprüngliche Farbe durch eine zweite XOR-Verknüpfung wieder her und erklären Sie dies an einem einzelnen Bit. Dieser feste Schlüssel dient nur als Lernmodell für Verschlüsselung.

06 Gleitkommazahlen kodieren und vergleichen | Combined

a) Kodieren Sie 13,25 als `float`: Wandeln Sie die Zahl ins Dualsystem um, normalisieren Sie die Mantisse und bestimmen Sie Vorzeichenbit, Exponent mit Bias 127 und die 23 gespeicherten Mantissenbits. Geben Sie die vollständigen 32 Bits an.

b) Welche Dezimalzahl ist in der folgenden `float`-Kodierung gespeichert? Die Abstände trennen Vorzeichen, Exponent und Mantissenfeld. Zeigen Sie die Rückrechnung.

```
1  10000001  10100000000000000000000
```

c) Berechnen Sie `double summe = 0.1 + 0.2;`. Geben Sie die Summe und den Wert von `summe == 0.3` aus. Ersetzen Sie den exakten Vergleich durch eine Prüfung, ob die absolute Abweichung höchstens `1e-12` beträgt. Verwenden Sie dafür `Math.abs()`. Warum muss eine geeignete Toleranz zum jeweiligen Anwendungsfall passen?

Combined: Zeichen und Zeichenketten

07 Eine Cäsar-Chiffre programmieren | Combined

Bei der Cäsar-Chiffre werden Buchstaben um eine feste Anzahl von Stellen verschoben. Nach Z geht es wieder bei A weiter. Wir verschlüsseln nur Großbuchstaben von A bis Z; alle anderen ASCII-Zeichen bleiben unverändert. Bei einer Verschiebung um 3 wird aus `ABC XYZ!` der Text `DEF ABC!`.

a) Bestimmen Sie mithilfe der ASCII-Tabelle die Codes von `'A'` und `'Z'`. Wie berechnen Sie aus einem Großbuchstaben seine Position von 0 bis 25 und aus einer solchen Position wieder einen `char`-Wert?

b) Lesen Sie einen Text und eine ganze Zahl `schluessel` von 0 bis 25 ein. Durchlaufen Sie den Text mit einer Schleife, `length()` und `charAt()`. Verwenden Sie `% 26` für den Übergang von Z zu A und einen passenden Cast. Bauen Sie den verschlüsselten Text in einer zunächst leeren `String`-Variablen auf und geben Sie ihn aus.

c) Testen Sie die Schlüssel 0, 3 und 25 mit `ABC XYZ! 123`. Prüfen Sie zusätzlich einen leeren Text und einen Text mit Kleinbuchstaben. Notieren Sie die erwarteten Ergebnisse vor dem Test.

d) Entschlüsseln Sie den erzeugten Text mit demselben Schlüssel und vergleichen Sie das Ergebnis mit dem ursprünglichen Text über `equals()`. Achten Sie darauf, dass bei der Rückverschiebung keine negativen Buchstabenpositionen entstehen. Warum darf der Vergleich der Textinhalte nicht mit `==` erfolgen?

08 Texte und eingelesene Zahlen | Combined

a) Bestimmen Sie für die Eingabe `Java` die vier Ausgaben. Erklären Sie, was die Variablen nach `text += "!";` referenzieren und weshalb der ursprüngliche Text erhalten bleibt.

```
String text = IO.readln("Text: ");
String kopie = text;
text += "!";
IO.println(text);
IO.println(kopie);
IO.println(text == kopie);
IO.println(kopie.equals("Java"));
```

b) Ermitteln Sie direkt mit dem Literal `"Programmierung"` seine Länge und den Teiltext von Index 0 bis ausschließlich Index 8. Verwenden Sie `length()` und `substring()`, ohne vorher eine Variable für den Text anzulegen.

c) Schreiben Sie ein Programm, das einen ganzzahligen Betrag von 0 bis 3 000 000 000 Cent einliest und als Euro und verbleibende Cent ausgibt. Verwenden Sie `Long.parseLong()`, Ganzzahldivision und Restwert. Für 1299 Cent soll die Ausgabe `12 Euro und 99 Cent` lauten. Testen Sie 0, 5, 99, 100, 1299 und 3 000 000 000 Cent. Warum brauchen Sie dafür keine Gleitkommarechnung?

Challenge: Lotto und Ganzzahlüberlauf

09 Lotto berechnen und den Überlauf erklären | Challenge

Wie viele verschiedene Tipps gibt es bei „6 aus 49“? Wählen Sie sechs unterschiedliche Zahlen aus 1 bis 49. Die Reihenfolge spielt keine Rolle; die Superzahl wird nicht berücksichtigt. Verwenden Sie die vorgegebene Formel:

$$Anzahl = \frac{49 \cdot 48 \cdot 47 \cdot 46 \cdot 45 \cdot 44}{6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1}$$

a) Berechnen Sie mit einem Taschenrechner den exakten Zähler, den Nenner und die Anzahl der Tipps. Prüfen Sie jeweils, ob der Wert in `int` passt.

b) Berechnen Sie dieselben Größen mit dem folgenden Java-Code. Vergleichen Sie die Ausgaben mit dem Taschenrechner. Bei welcher Multiplikation tritt erstmals ein Überlauf auf? Meldet Java diesen automatisch?

```
int ersteFuenf = 49 * 48 * 47 * 46 * 45;
int zaehler = ersteFuenf * 44;
int nenner = 6 * 5 * 4 * 3 * 2 * 1;
int anzahl = zaehler / nenner;
IO.println(zaehler);
IO.println(anzahl);
```

c) Stellen Sie das mathematisch exakte Produkt `ersteFuenf * 44` vollständig im Dualsystem dar. Markieren Sie die Bits oberhalb der unteren 32 Bits, die herausfallen, und geben Sie ihre Stellenwerte an. Welcher Dezimalwert ist in den verbleibenden 32 Bits gespeichert?

d) Berechnen Sie für den Zähler und für die Anzahl der Tipps jeweils die genaue Abweichung als „Java-Ergebnis minus exakter Wert“. Begründen Sie den Verlust beim Zähler anhand der Bits. Warum stellt die anschließende Division den korrekten Wert nicht wieder her?

e) Welche der folgenden Varianten berechnet den Zähler korrekt? Begründen Sie dies mit dem Zeitpunkt des Casts und prüfen Sie es in Java. Schreiben Sie anschließend eine korrekte Version der gesamten Berechnung mit `long`. Warum braucht der gezeigte Rechenweg einen größeren Wertebereich, obwohl die endgültige Anzahl der Tipps in `int` passt?

```
long z1 = ersteFuenf * 44;
long z2 = (long) (ersteFuenf * 44);
long z3 = (long) ersteFuenf * 44;
```

Challenge: Rechenwege und Gleitkommarundung

10 Zwei Rechenwege mit `float` vergleichen | Challenge

Nach dem Distributivgesetz sind `x * (y + z)` und `x * y + x * z` mathematisch gleich. Untersuchen Sie, ob das auch für die folgenden Berechnungen mit `float` gilt. Alle drei Ausgangswerte sind exakt als `float` darstellbar.

```
float x = 3.5f;
float y = -8_388_608f;
float z = 8_388_609f;
float geklammert = x * (y + z);
float ausmultipliziert = x * y + x * z;
IO.println(geklammert);
IO.println(ausmultipliziert);
```

a) Bestimmen Sie zuerst von Hand das mathematisch exakte Ergebnis beider Rechenwege. Führen Sie dann das Programm aus. Notieren Sie die beiden Ausgaben und ihre Abweichungen vom exakten Ergebnis.

b) Untersuchen Sie die Multiplikation `x * z`: Berechnen Sie ihr exaktes Produkt im Dezimal- und Dualsystem und normalisieren Sie die binäre Mantisse. Markieren Sie die 24 signifikanten Bits einschließlich der führenden 1 sowie alle zusätzlichen Bits, die nicht mehr in die Mantisse passen.

c) Schieben Sie das Komma entsprechend dem Exponenten zurück und bestimmen Sie den Wert der zusätzlichen Bits. Ermitteln Sie die benachbarten darstellbaren `float`-Werte unterhalb und oberhalb des exakten Produkts sowie beide Abstände. Welcher Nachbar liegt näher? Zeigen Sie beim Aufrunden den Übertrag im beibehaltenen Bitteil. Um wie viel erhöht ein Schritt hier den Zahlenwert?

d) Geben Sie die endgültige 32-Bit-Kodierung von `x * z` und die genaue Rundungsabweichung an. Untersuchen Sie auch `y + z` und `x * y`. Erklären Sie damit die unterschiedlichen Endergebnisse aus a).

e) Ersetzen Sie nur `x = 3.5f` durch `x = 2.5f`. Bestimmen Sie erneut das exakte Endergebnis und die beiden Java-Ausgaben. Wird das Produkt `x * z` jetzt ab- oder aufgerundet? Begründen Sie dies anhand seiner beiden Nachbarwerte; eine erneute vollständige Bitkodierung ist nicht erforderlich.

f) Führen Sie den ersten Fall mit `double` statt `float` und passenden Literalen erneut aus. Welche Ergebnisse entstehen? Erklären Sie, warum ein nachträglicher Cast des bereits berechneten `ausmultipliziert` nach `double` die vorher verlorene Genauigkeit nicht zurückbringt.

Zur Kontrolle nach der Handrechnung: IEEE-754-Konverter von Harald Schmidt